Fuzz Testing Report — Pizza Order

Application: pizza_order.py • SETAPESU26/L6_L7_24_pizza_order, Lab 7

Submitted by

Akshat Agarwal

SRN: PES1UG24AM024

11 November 2026 • Repository: github.com/SETAPESU26/L6_L7_24_pizza_order (Lab 7)

Environment: Python 3.12.3, pytest 9.2.0, Hypothesis 6.170.15, run on Linux. The handout calls the application processor.py and the test file test_processor.py; in the repo they are pizza_order.py and test_pizza_order.py — the same two files, and this report uses the repo names. Identical copies under the handout names are included in the submission zip.

Summary: the buggy application crashed on ordinary malformed input in all three functions. Hypothesis fuzzing found minimised failing inputs for every function (18 of 21 tests failed, 3 passed). After hardening, the same 21 tests all pass — twice in a row — and the application handles the same invalid inputs without crashing, while valid inputs return exactly the same results as before.

1. What fuzzing and Hypothesis are

Fuzz testing feeds random, unexpected or malformed data to a program to find bugs that hand-written tests miss. Unit tests check what the author thought might go wrong; a fuzzer finds what nobody imagined. Hypothesis is a property-based fuzzer for Python: instead of one example per test, you state a property the function must always satisfy (here, 'it must not crash, and its result must stay in a sane range'), and Hypothesis generates a hundred random inputs per test, then shrinks any failing input to the smallest example that still fails. That shrinking is what makes the failing inputs in §5 so useful — they are not random strings, they are the minimal proof of each bug.

Each Hypothesis test in this lab uses max_examples=100, so the suite exercises each property on 100 generated inputs, plus the deterministic tests in §4.

2. Application under test

pizza_order.py (the handout's processor.py) has three functions and a small interactive main() that prompts for one input per function. The buggy version validated nothing:

Function	Buggy behaviour	Valid example
toppings_price(toppings_csv)	Splits on ',' and looks every token up in TOPPINGS, with no check. Unknown or empty tokens raise KeyError; None raises AttributeError.	cheese,olives -> 70
pizza_inches(size)	Looks the stripped, upper-cased size up	S -> 8, m -> 10, ' L ' -> 12

	in SIZES, with no check. Unknown or empty sizes raise KeyError; None raises AttributeError.	
split_bill(text)	Unpacks text.split('/') into exactly two parts and does int(total) / int(people). Wrong part count raises ValueError, non-numeric parts raise ValueError, people = 0 raises ZeroDivisionError, None raises AttributeError.	900/3 -> 300.0

The starter test file was itself broken: its three test functions had empty bodies, so pytest could not even collect it — IndentationError: expected an indented block after function definition on line 5. Completing those three functions was the first part of Step 3.

3. Step 2 — Running the buggy application

Valid inputs worked exactly as intended: cheese,olives gave Toppings Price 70, S gave 8 inches, and 900/3 gave Per Person 300.0. The first invalid input stopped the program: cheese,xyz crashed with KeyError: 'xyz' at toppings_price, line 7, before the other two functions were even reached. A systematic bug demo (calling each function directly with edge inputs) found crashes in all three functions, in five distinct exception types:

Function	Input	Buggy result
toppings_price	''	KeyError: ''
toppings_price	'cheese,xyz'	KeyError: 'xyz'
toppings_price	'cheese,,olives'	KeyError: '' (empty token)
toppings_price	None	AttributeError: 'NoneType' has no attribute 'split'
pizza_inches	''	KeyError: ''
pizza_inches	'X'	KeyError: 'X'
pizza_inches	None	AttributeError: 'NoneType' has no attribute 'strip'
split_bill	'' / '900'	ValueError: not enough values to unpack
split_bill	'900/0'	ZeroDivisionError: division by zero
split_bill	'900/three'	ValueError: invalid literal for int()
split_bill	'900.5/3'	ValueError: invalid literal for int()
split_bill	'900/3/1'	ValueError: too many values to unpack
split_bill	None	AttributeError: 'NoneType' has no attribute 'split'
split_bill	'900/-3' / '-100/2'	No crash, but -300.0 / -50.0 — a negative per-person share

Two inputs already worked correctly and were preserved as-is by the fix: case and surrounding spaces are tolerated (CHEESE, Olives -> 70; ' S ' -> 8). The bugs are therefore all in input validation, not in the valid-path arithmetic.

4. Step 3 — The completed Hypothesis test file

File: test_pizza_order.py (the handout's test_processor.py). 21 collected test items: 7 Hypothesis property tests, 5 deterministic test functions, and 9 parametrised invalid-input cases for split_bill.

4.1 The three starter tests, completed

Each takes st.text() | st.none() — any text, or None — and asserts a no-crash property: toppings_price returns an int >= 0; pizza_inches returns one of (0, 8, 10, 12); split_bill returns a float >= 0. These are the tests the handout asked for; on the buggy code all three failed.

4.2 Extra fuzzing with richer strategies

Test	Strategy	Property checked
test_toppings_price_csv_lists	Lists of 0–6 random texts, joined with ','	Result equals the sum of the known toppings only; unknown tokens contribute 0
test_pizza_inches_short_text	Text of length 0–3	Result is always in (0, 8, 10, 12)
test_split_bill_structured	Integers total in [-10000, 10000], people in [-10, 10], as 'total/people'	If people > 0 and total >= 0, result is round(total/people, 2); otherwise 0.0 — never a crash
test_split_bill_any_text	Any text	Result is always a float >= 0

4.3 Deterministic tests

Valid examples (toppings_price 'cheese,olives' = 70, 'CHEESE, Olives ' = 70, 'paneer' = 50; pizza_inches S/m/' L ' = 8/10/12; split_bill '900/3' = 300.0, '1000/4' = 250.0) prove valid behaviour is preserved. Invalid-input tests pin down the exact edge cases Hypothesis minimised: toppings_price '' / None / 'xyz' -> 0, 'cheese,xyz' -> 40, 'cheese,,olives' -> 70; pizza_inches '' / 'X' / None -> 0; split_bill '' / '900' / '900/0' / '900/three' / '900.5/3' / '900/3/1' / None / '900/-3' / '-100/2' -> 0.0.

5. Step 4 — Fuzz tests against the buggy code: 18 failed, 3 passed

Command: python -m pytest test_pizza_order.py -v. Result: 18 failed, 3 passed in 6.44 s. The only three passing tests were the valid-example tests — every test that touched an invalid input failed. Hypothesis minimised each failure to the smallest input that still fails:

Test	Minimised failing input	Failure
test_toppings_price_no_crash	s = ''	KeyError: ''
test_toppings_price_no_crash	s = None	AttributeError: no attribute 'split'
test_pizza_inches_no_crash	s = ''	KeyError: ''
test_pizza_inches_no_crash	s = None	AttributeError: no attribute 'strip'
test_split_bill_no_crash	s = '/'	ValueError: invalid literal for int(): ''
test_split_bill_no_crash	s = ''	ValueError: not enough values to unpack
test_split_bill_no_crash	s = None	AttributeError: no attribute 'split'
test_toppings_price_csv_lists	tokens = [] (csv = '')	KeyError: ''
test_pizza_inches_short_text	s = ''	KeyError: ''
test_split_bill_structured	total = 0, people = 0	ZeroDivisionError: division by zero
test_split_bill_structured	total = 1, people = -1	AssertionError: -1.0 != 0.0
test_split_bill_any_text	s = '/'	ValueError: invalid literal for int(): ''

Two findings go beyond a simple crash. First, Hypothesis found two distinct failures inside single tests (reported as ExceptionGroups) — for example, toppings_price fails on both '' and None, which are different bugs (missing key vs missing method). Second, split_bill('900/-3') and split_bill('-100/2') do not crash, but return -300.0 and -50.0: a negative per-person share, which is wrong even though it is not an exception. Both are fixed in §6 by rejecting non-positive people counts and negative totals.

6. Step 5 — Fixing the application

Hardening rules, chosen so no text or None input can crash a function while valid inputs return exactly what they returned before:

Function	Fix	Invalid input -> fixed result
toppings_price	Reject non-strings (return 0). Look tokens up with 'if name in TOPPINGS' instead of direct indexing, so unknown or empty tokens are ignored.	'' -> 0; None -> 0; 'cheese,xyz' -> 40; 'cheese,,olives' -> 70

pizza_inches	Reject non-strings (return 0). Use SIZES.get(..., 0) instead of direct indexing.	'' -> 0; 'X' -> 0; None -> 0
split_bill	Reject non-strings (return 0.0). Require exactly two '/' parts, parse each with int() inside try/except (ValueError, OverflowError) -> 0.0, and require total >= 0 and people > 0.	'' / '900' / '900/3/1' -> 0.0; '900/0' -> 0.0; '900/three' / '900.5/3' -> 0.0; None -> 0.0; '900/-3' / '-100/2' -> 0.0
main()	Wrap each input() in try/except (EOFError, KeyboardInterrupt) and print 'No input given.' instead of crashing when stdin is closed.	Empty stdin -> 'No input given.', exit cleanly

Valid behaviour is unchanged, verified directly: cheese,olives still gives 70, S still gives 8, 900/3 still gives 300.0. The fix is defensive, not permissive: it does not accept decimal totals ('900.5/3' -> 0.0) or extra parts, because the stated format is integer total / integer people.

A design note, stated honestly: returning 0 / 0.0 for invalid input is a choice. An alternative is to raise a clear ValueError, but that would still crash the no-crash property tests the handout specifies, and the interactive program would still stop on the first bad input. The chosen safe defaults keep the program running and are documented in the fixed file's docstrings.

7. Step 6 — Re-running the tests on the fixed code: 21 passed

Command: python -m pytest test_pizza_order.py -v. Result: 21 passed in 0.90 s. A second run, to check the randomised suite is stable rather than lucky, also gave 21 passed in 0.68 s. Every one of the 18 tests that failed on the buggy code now passes, and the 3 valid-example tests still pass.

The fixed application was also re-run interactively with the same inputs that crashed it in Step 2: 'cheese,xyz' now gives Toppings Price 40, 'X' gives Size in Inches 0, and '900/0' gives Per Person 0.0 — the program completes all three prompts instead of stopping at the first crash.

8. Screenshots

```
$ printf 'cheese,olives\nS\n900/3\n' | python pizza_order.py   (valid inputs)

==== Running toppings_price ====
Enter toppings as CSV (e.g. cheese,olives): Toppings Price: 70

==== Running pizza_inches ====
Enter pizza size (S/M/L): Size in Inches: 8

==== Running split_bill ====
Enter total and people (e.g. 900/3): Per Person: 300.0
```

Figure 1 — Step 2, buggy application with valid inputs: 70, 8, 300.0.

```
$ printf 'cheese,xyz\n' | python pizza_order.py   (Step 2, buggy application)

==== Running toppings_price ====
Enter toppings as CSV (e.g. cheese,olives): Traceback (most recent call last):
  File "/home/hatch/workspace/lab7-fuzz/work/pizza_order.py", line 34, in <module>
    main()
  File "/home/hatch/workspace/lab7-fuzz/work/pizza_order.py", line 20, in main
    result = toppings_price(user_input)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/hatch/workspace/lab7-fuzz/work/pizza_order.py", line 7, in toppings_price
    total += TOPPINGS[t.strip().lower()]
             ~~~~~~~~^^^^^^^^^^^^^^^^^^^
KeyError: 'xyz'
```

Figure 2 — Step 2, buggy application with 'cheese,xyz': KeyError: 'xyz'.

```
$ python -m pytest test_pizza_order.py -v   (Step 4, completed tests vs BUGGY code)

test_pizza_order.py::test_toppings_price_no_crash FAILED                 [  4%]
test_pizza_order.py::test_pizza_inches_no_crash FAILED                   [  9%]
test_pizza_order.py::test_split_bill_no_crash FAILED                     [ 14%]
test_pizza_order.py::test_toppings_price_csv_lists FAILED                [ 19%]
test_pizza_order.py::test_pizza_inches_short_text FAILED                 [ 23%]
test_pizza_order.py::test_split_bill_structured FAILED                   [ 28%]
test_pizza_order.py::test_split_bill_any_text FAILED                     [ 33%]
test_pizza_order.py::test_toppings_price_valid_examples PASSED           [ 38%]
test_pizza_order.py::test_toppings_price_invalid_returns_zero FAILED     [ 42%]
test_pizza_order.py::test_pizza_inches_valid_examples PASSED             [ 47%]
test_pizza_order.py::test_pizza_inches_invalid_returns_zero FAILED       [ 52%]
test_pizza_order.py::test_split_bill_valid_examples PASSED               [ 57%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[] FAILED       [ 61%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[900] FAILED    [ 66%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[900/0] FAILED  [ 71%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[900/three] FAILED [ 76%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[900.5/3] FAILED [ 80%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[900/3/1] FAILED [ 85%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[None] FAILED   [ 90%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[900/-3] FAILED [ 95%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[-100/2] FAILED [100%]

======================== 18 failed, 3 passed in 6.44s ========================
```

Figure 3 — Step 4, completed tests vs buggy code: 18 failed, 3 passed.

```
$ printf 'cheese,xyz\nX\n900/0\n' | python pizza_order.py   (Step 5, FIXED application)

==== Running toppings_price ====
Enter toppings as CSV (e.g. cheese,olives): Toppings Price: 40

==== Running pizza_inches ====
Enter pizza size (S/M/L): Size in Inches: 0

==== Running split_bill ====
Enter total and people (e.g. 900/3): Per Person: 0.0
```

Figure 4 — Step 5, fixed application with the same invalid inputs: 40, 0, 0.0, no crash.

```
$ python -m pytest test_pizza_order.py -v   (Step 6, completed tests vs FIXED code)

test_pizza_order.py::test_toppings_price_no_crash PASSED                [  4%]
test_pizza_order.py::test_pizza_inches_no_crash PASSED                  [  9%]
test_pizza_order.py::test_split_bill_no_crash PASSED                    [ 14%]
test_pizza_order.py::test_toppings_price_csv_lists PASSED               [ 19%]
test_pizza_order.py::test_pizza_inches_short_text PASSED                [ 23%]
test_pizza_order.py::test_split_bill_structured PASSED                  [ 28%]
test_pizza_order.py::test_split_bill_any_text PASSED                    [ 33%]
test_pizza_order.py::test_toppings_price_valid_examples PASSED          [ 38%]
test_pizza_order.py::test_toppings_price_invalid_returns_zero PASSED    [ 42%]
test_pizza_order.py::test_pizza_inches_valid_examples PASSED            [ 47%]
test_pizza_order.py::test_pizza_inches_invalid_returns_zero PASSED      [ 52%]
test_pizza_order.py::test_split_bill_valid_examples PASSED              [ 57%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[] PASSED      [ 61%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[900] PASSED   [ 66%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[900/0] PASSED [ 71%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[900/three] PASSED [ 76%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[900.5/3] PASSED [ 80%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[900/3/1] PASSED [ 85%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[None] PASSED  [ 90%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[900/-3] PASSED [ 95%]
test_pizza_order.py::test_split_bill_invalid_returns_zero[-100/2] PASSED [100%]

============================== 21 passed in 0.90s ==============================
```

Figure 5 — Step 6, completed tests vs fixed code: 21 passed.

9. Interpretation, limits, and conclusion

- What fuzzing proved here: all three functions crashed on trivially reachable inputs (an empty string, None, an unknown word, a zero divisor). Manual tests had missed them because the valid path worked perfectly — the program looked finished.
- What the minimised examples add: they turn 'it crashes sometimes' into a precise bug list, with the smallest input for each bug, which is what made the fix in §6 a short, targeted change rather than a rewrite.
- What fuzzing does not prove: that the functions are correct. 100 random inputs per property, plus deterministic tests, is strong evidence against crashes, but it is not a proof — and the safe defaults (0 / 0.0) are a design choice, not a business rule. A different valid specification (for example, rejecting unknown toppings with an error message) would need different tests.
- Conclusion: the lab is complete. The buggy application was run and crashed (Step 2), Hypothesis tests were completed and failed on it with minimised counterexamples (Steps 3–4, 18 failed / 3 passed), the application was hardened (Step 5), and the same tests now pass on the fixed code (Step 6, 21 passed, twice).

Appendix — How to reproduce

Install: pip install hypothesis pytest (tested with Hypothesis 6.170.15, pytest 9.2.0, Python 3.12.3). Run the application: python pizza_order.py. Run the tests: python -m pytest test_pizza_order.py -v. Deliverables in the submission zip: this report (PDF + DOCX), the fixed pizza_order.py (also copied as processor.py, the handout's name), the completed test_pizza_order.py (also copied as test_processor.py), the Screenshots/ folder, and the terminal logs in logs/.